System design and architecture · v2 (as built + target) · 29 October 2026 · Prototype v1 at github.com/devashishb-rms/AutoJopps

AutoJopps: System Design and Architecture

Every component, module, data store and external service in the implementation, how they interact, and the target cloud deployment on AWS that the same code grows into.

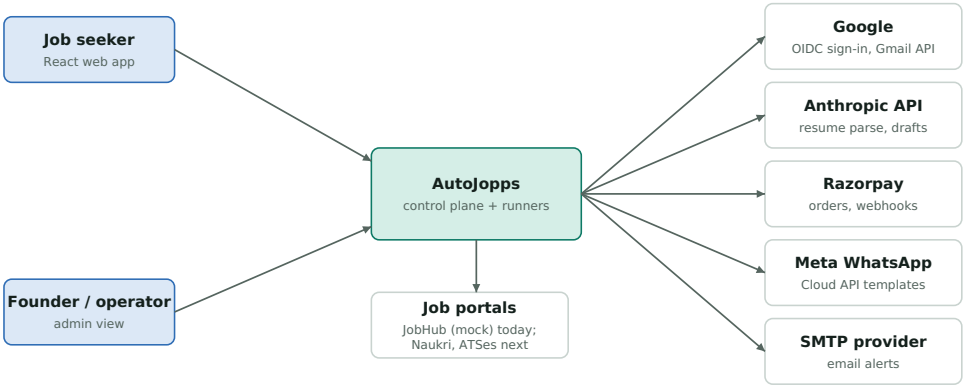
> **The name. AutoJopps** (pronounced auto-J-opps) is short for **Automated New Job Opportunities and Smooth Job-Search Operations and Job Hops**: receive frequent new job opportunities through automated, fast job applies. The product may also be called or named **AutoJobOpps**.

> **Two views in one document.** Sections 2-8 describe the system **as implemented** in Prototype v1 (one FastAPI process, SQLite, local files, Playwright). Section 9 maps each piece to the **target** AWS services; section 10 lists third-party services in both.

1. Architecture principles

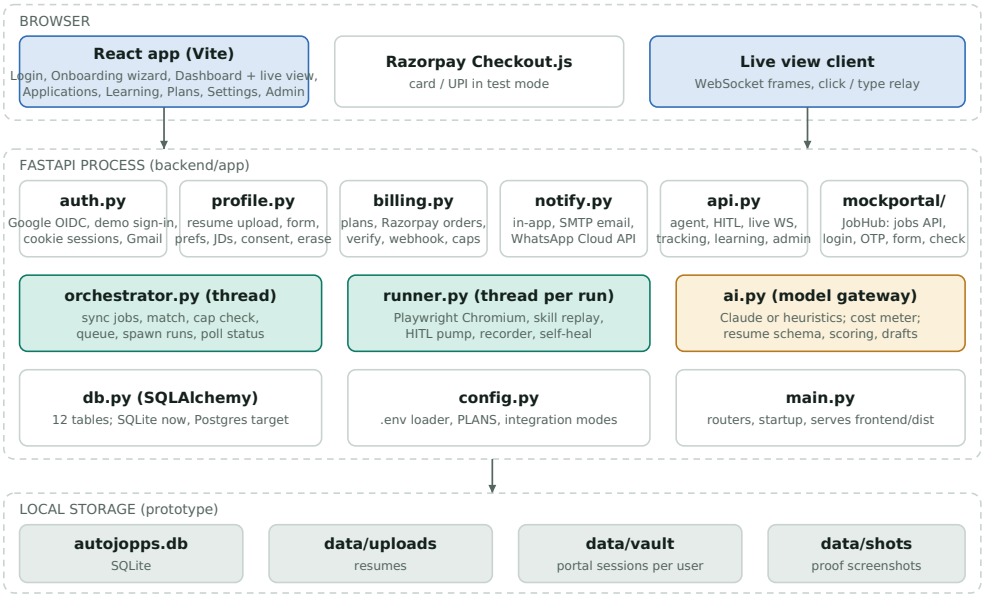
1. **One orchestrator, many small portal adapters.** A skill (list of steps with several locators per target) is the adapter format; the runner replays skills and self-heals.
2. **Human help is a workflow state**, not an exception path: *needs_human* and *awaiting_approval* with resume at the same step.
3. **The human signs in; the bot never types a Google password.** Only the resulting portal session is stored in a per-user vault.
4. **Each run gets its own browser context**, created from the vault and closed after the run.
5. **Caps and billing in one place** (entitlements), driven by payment events.
6. **Cost per application is a metric from day one** (browser share + model tokens + messaging).

2. System context



External actors and services. JobHub is a mock portal served by the app for demos; real portals are added as adapters.

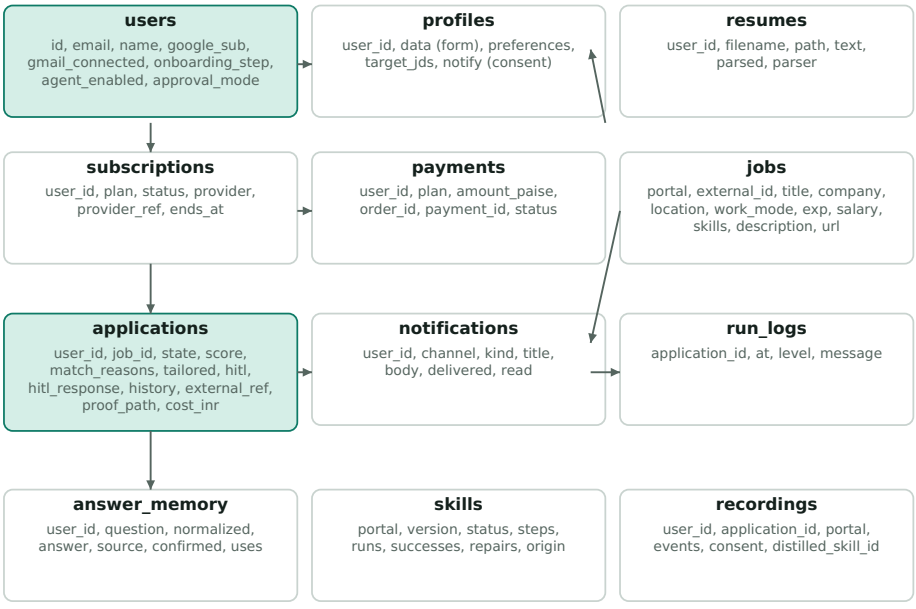
3. Components and modules (as implemented)



Prototype v1: one process. Threads for the orchestrator and for each application run.

Module	Responsibility	Key interfaces
`frontend/` React 18 + Vite + react-router	9 screens: Login, Onboarding (8 steps), Dashboard (agent, HITL card, live view), Applications (list + detail), Learning, Plans, Settings, Admin	REST under `/api`, WebSocket `/api/live/{id}`; built to `frontend/dist` and served by FastAPI
`auth.py`	Google OIDC code flow, demo sign-in, signed cookie sessions (itsdangerous), Gmail connect (scope flag) and recent-mail read, forwarding-alias fallback	`/api/auth/*`; `current_user` dependency
`profile.py`	Resume upload (pypdf, python-docx), parse, Naukri-style form, preferences, target JDs, notification consent, settings, DPDP erasure	`/api/profile/*`
`ai.py`	Model gateway: Claude (Anthropic SDK) when a key exists, deterministic heuristics otherwise; resume schema; keyword scoring with hard filters; answer drafting; token cost meter	`parse_resume`, `match_score`, `draft_answer`, `COST_LOG`
`billing.py`	Plans, Razorpay orders (REST), HMAC signature verification, webhook, demo activation, subscription window, usage and caps	`/api/billing/*`; `can_apply`, `usage`
`orchestrator.py`	Background thread: job sync, per-user matching, cap check, queue next, spawn runner thread (one per user), poll portal status, recover after restart	`start()`, `tick()`, `run_one_now`
`runner.py`	Playwright Chromium per run; vault storage state; skill replay with multi-locator `locate` and repair logging; question answering (memory -> profile -> Claude -> human); HITL pump (`wait_for_human`); recorder; proof; trial skill filing	`run_application(id)`; `LIVE` registry of `LiveSession`
`api.py`	Agent control, HITL responses, demo helpers, live-view WebSocket, tracking (list, detail, proof), learning (answers, skills, recordings, distil, promote), admin stats	`/api/agent/*`, `/api/live/{id}`, `/api/applications*`, `/api/learning*`, `/api/admin/stats`
`notify.py`	In-app notifications; SMTP email; WhatsApp Cloud API template messages; consent-gated fan-out with delivery status	`notify()`, `/api/notifications`
`mockportal/`	JobHub: 24 seeded jobs, login + OTP, application form, screening questions, human check, confirmation reference, status simulation, chaos switch	`/portal/*`, `/portal/api/jobs`, `/portal/api/status/{ref}`
`db.py`, `config.py`, `main.py`	SQLAlchemy models and sessions; .env loader, plan table, integration modes; app wiring and SPA serving	

4. Data model



One file-based vault entry per (user, portal) holds the browser storage state; it is not a table.

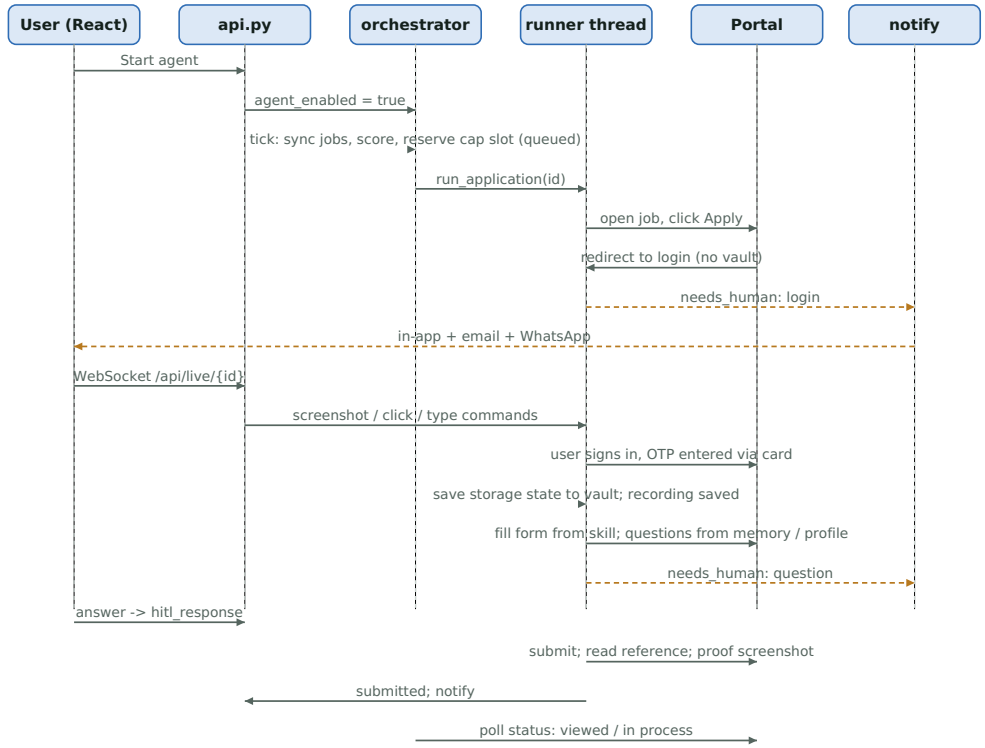
Twelve tables in SQLite (SQLAlchemy); JSON columns hold the flexible parts (form, preferences, hitl, history, steps).

Store	Prototype	Target	Contents
Relational	SQLite file	RDS PostgreSQL + pgvector	All tables; embeddings for JD similarity

Resumes, proofs, recordings	`data/uploads`, `data/shots`	S3 with SSE-KMS, lifecycle rules	Per-tenant prefixes
Session vault	`data/vault/u{id}_{portal}.json` (plain)	S3 or DynamoDB, encrypted with a per-tenant KMS key	Playwright storage state (cookies, local storage) only
Queues, locks, caps	In-process (thread lock, DB rows)	ElastiCache Redis	Run queue, per-user lock, cap counters, HITL pub/sub
Event log	`run_logs` + `applications.history`	Same tables + CloudWatch / Kinesis	Audit trail per application

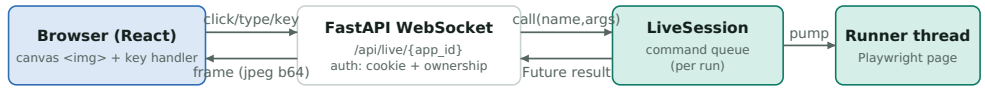
5. Key flows

5.1 First application with human help



The first run on a portal: sign-in and OTP by the human, then form filling, one unusual question, submission and status polling. Amber arrows are human-help notifications.

5.2 Live view and input relay



/wright objects are bound to the runner thread, so every live-view action is executed by that thread via the queue (0.5 s poll, 0.8 s HITL respons

How the user sees and drives the bot's own browser.

5.3 Self-heal and skill versions

1. Each skill step names its target with several locators in priority order (`id`, `name`, `label`, `css`, `text`).
2. `runner.locate` tries them in order; when the first fails but a later one works, it logs a repair and remembers the working locator.
3. After a successful run with repairs, a new skill version is filed as *trial* with the repaired locator promoted to first place; counters (`runs`, `successes`, `repairs`) are updated on the active version.
4. The operator promotes a trial to active (or it is promoted automatically after N clean runs in the target design); the old version is retired and can be restored.
5. Recorded demonstrations are distilled into *draft* skills by mapping typed fields to profile bindings and clicks to steps; passwords are masked at capture.

6. API surface

Area	Endpoints

Auth	`GET /api/auth/config`, `/google/start`, `/google/callback`, `POST /demo`, `GET /me`, `POST /logout`, `POST /gmail/demo-connect`, `GET /gmail/recent`
Profile	`POST /api/profile/resume`, `GET /api/profile`, `PUT /form`, `/preferences`, `/target-jds`, `/notify`, `/step`, `/settings`, `POST /delete-my-data`
Billing	`GET /api/billing/plans`, `/status`, `POST /demo-activate`, `/order`, `/verify`, `/webhook`, `/cancel`
Agent and HITL	`GET /api/agent/status`, `POST /toggle`, `/discover`, `/apply/{id}`, `/hitl/{id}`, `/hitl/{id}/login-done`, `/hitl/{id}/demo-login`, `GET /demo-phone`, `POST /chaos`
Live view	`WS /api/live/{application_id}`: server sends `frame` / `busy` / `ended`; client sends `click`, `type`, `key`, `record`
Tracking	`GET /api/applications`, `/applications/{id}`, `/applications/{id}/proof`, `GET /api/notifications`, `POST /read-all`
Learning	`GET /api/learning`, `POST /answers`, `PUT /answers/{id}`, `DELETE /answers/{id}`, `POST /recordings/{id}/distill`, `PUT /skills/{id}`
Admin	`GET /api/admin/stats`
Mock portal	`/portal/*` pages, `/portal/api/jobs`, `/portal/api/status/{ref}`, demo helpers `/_demo/*`

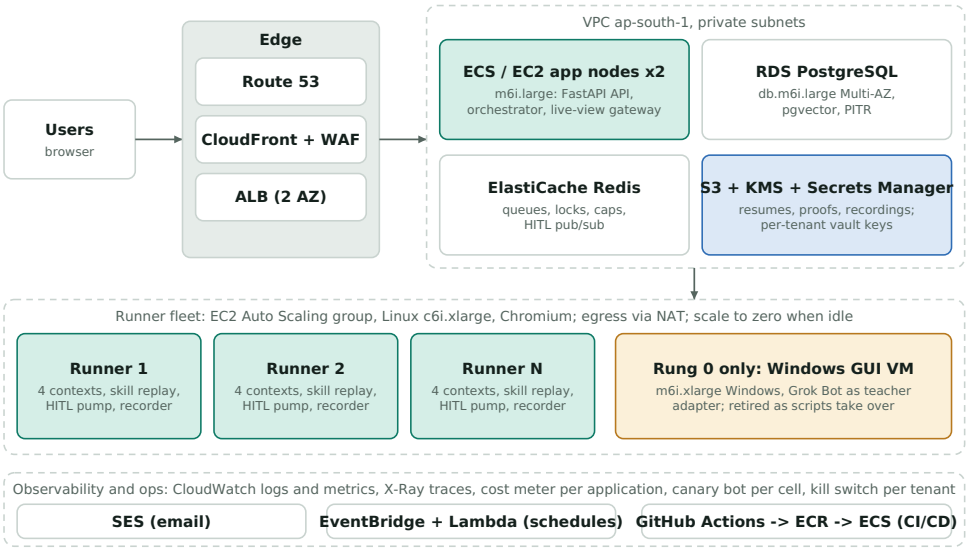
7. Security

Control	Prototype v1	Target
Identity	Google OIDC with state check; signed HttpOnly cookie (14 days); demo sign-in for local use	OIDC only in production; short sessions, refresh, device list
Credentials	Google / portal passwords never stored; portal session only, in a per-user vault file	Vault encrypted with per-tenant KMS keys; access logged; rotated on logout
Live view	WebSocket authorised by cookie and application ownership; frames are JPEG, no DOM exposed	Signed single-use links with expiry; rate limits; recording consent per session
Payments	Razorpay signature (HMAC-SHA256) and webhook signature verified; secrets in .env	Secrets Manager; idempotent webhooks; GST invoices
Model calls	API keys under commercial terms; prompts carry only the needed profile fields	Per-tenant cost meter and kill switch; PII minimisation in prompts
Isolation	New browser context per run; threads per run; SQLite	Runner VM per cell; per-tenant prefixes and keys; optional silo cell
Audit	History and run log per application; notification delivery status	Immutable event log; retention policy

8. Privacy (DPDP) posture

- **Notice and consent:** alerts consent captured with timestamp; recording consent at capture; a plain-language notice is still to be written.
- **Purpose limitation and minimisation:** resume parsed to a fixed schema; prompts carry only the fields needed; passwords never stored.
- **Erasure:** `POST /api/profile/delete-my-data` removes resume files, profile, answers, recordings, applications, run logs, proof screenshots, portal-session vault files, notifications, subscriptions and payments.
- **Processors:** Google, Anthropic, Razorpay, Meta (WhatsApp), the SMTP provider, AWS (target). Contracts and a processor register are needed before launch.
- **Breach:** the event log supports investigation; the notification runbook (Data Protection Board and affected users) is planned.
- **Retention:** recordings 30 days after distillation (proposed); proofs for the subscription period plus 90 days (proposed).

9. Target cloud deployment (AWS, Mumbai)



Production shape for the same code: stateless app nodes, managed data services, an autoscaling runner fleet.

Component (prototype)	AWS service (target)	Sizing to start	Notes
FastAPI process	ECS on EC2 or plain EC2, m6i.large x2 behind ALB	2 nodes, 2 AZ	API, orchestrator, live-view gateway; orchestrator leader-elected via Redis lock
Runner threads + local Chromium	EC2 Auto Scaling group, c6i.xlarge Linux	2 + 1 spare; ~180 users per node (scripted)	Playwright; 4 contexts per node; NAT egress; scale to zero overnight
Grok Bot teacher (optional)	EC2 m6i.xlarge Windows	as needed	Only until scripted adapters cover a portal
SQLite	RDS PostgreSQL db.m6i.large Multi-AZ + pgvector	100 GB gp3	PITR backups
In-process locks / queues	ElastiCache Redis (2 nodes)	cache.t4g.small	Queues, caps, HITL pub/sub
data/ folders	S3 (SSE-KMS) + KMS + Secrets Manager	-	Per-tenant prefixes and keys
Static UI	CloudFront + WAF + Route 53	-	TLS, bot and abuse protection
SMTP	SES	-	Alerts and receipts
Status polling thread	EventBridge schedule + Lambda	-	Portal status, cleanup, retention
print / run log	CloudWatch Logs and Metrics, X-Ray	-	Cost per application dashboard, alarms
Manual deploy	GitHub Actions -> ECR -> ECS	-	Blue/green

Estimated run cost (from the cost estimate, on-demand Mumbai prices): about $0.9k / month at 100 users, $2.0k at 500, $6.3k at 2,000 on scripted adapters, including model and messaging.

10. External services

Service	Used for	Mode without key	Terms to note
Google Identity / Gmail API	Sign-in; optional read-only recruiter-mail signals	Demo sign-in; forwarding alias	Gmail is a restricted scope: verification + CASA for public use; Google blocks automated sign-ins
Anthropic API (Claude)	Resume parsing, answer drafts, tailoring	Heuristic parser, templates	Commercial terms via API key; no consumer subscriptions as the engine
Razorpay	Orders, checkout, webhooks (Subscriptions next)	Demo activation	Test mode keys; UPI AutoPay limits to confirm
Meta WhatsApp Cloud API	Human-help alerts	Logged, in-app only	Business-initiated messages outside 24 h need approved utility templates
SMTP / SES	Email alerts	Logged, in-app only	SPF / DKIM for deliverability
Playwright + Chromium	Browser automation	-	Installed per runner
Job portals	Discovery and applying	JobHub mock	Each adapter respects the portal’s terms; LinkedIn automation off by default

11. Operations and observability

- Integration modes are reported at startup and in the UI (`settings.modes()`), so a demo never silently depends on a missing key.
- Run log per application; history with timestamps; notification delivery status per channel.
- Cost meter: browser share per application plus model tokens priced in `ai.py` (one place to update list prices).
- Restart safety: in-flight runs are marked *failed* on startup and can be retried; cap slots are not double counted.
- Target: canary bot per cell replays active skills daily on test accounts; per-tenant kill switch when cost per application exceeds a threshold.

12. Known gaps and technical debt

Gap	Impact	Plan
SQLite and in-process state	Single node only	Postgres + Redis (v1.1)
Vault stored as plain JSON	Session theft if the disk is read	KMS-encrypted vault (v1.1)
Admin view open to any signed-in user	Prototype only	Role claim on the user (v1.1)
One runner thread per user; tier runner counts not scheduled	Premium / Ultra get no parallelism yet	Runner pool with tier lanes (v1.2)
Only the mock portal has a skill	No real applications yet	Naukri + one ATS adapter (v1.1)
One-time Razorpay orders	No auto-renewal	Razorpay Subscriptions (v1.1)
Keyword matching	Coarser ranking	pgvector embeddings (v1.2)

Prepared 29 October 2026 from commit bcbd67e. Sizing and costs are estimates from the cost-estimate document; prices were checked on 28 October 2026.